

Week 11

Data Rendering

Load and render dynamic content from external data sources.



Learning outcomes

Load and render dynamic content from external data sources.
Explain the core programming/design concept for this week
Read and modify key code patterns
Test the weekly project increment
Prepare evidence for GitHub submission

Today's problem



Why it matters

Data rendering is a pipeline: fetch, validate, transform, render and handle failure.

Project before -> after

Before Week 11

Week 10 persisted favourites and preferences using Web Storage.

After Week 11

The site renders data-driven content more reliably and connects filters to the user interface.

What we build this week



Review the data shape
Write renderCard() for one item
Render all matching items
Add filter controls and empty states
Connect favourites to rendered cards

Core principle

Data rendering is a pipeline: fetch, validate, transform, render and handle failure.

Keep in mind

A good web developer can point to the code, the browser result, and the user reason behind the decision.

Key concepts

Concept 1

Arrays can be filtered before rendering

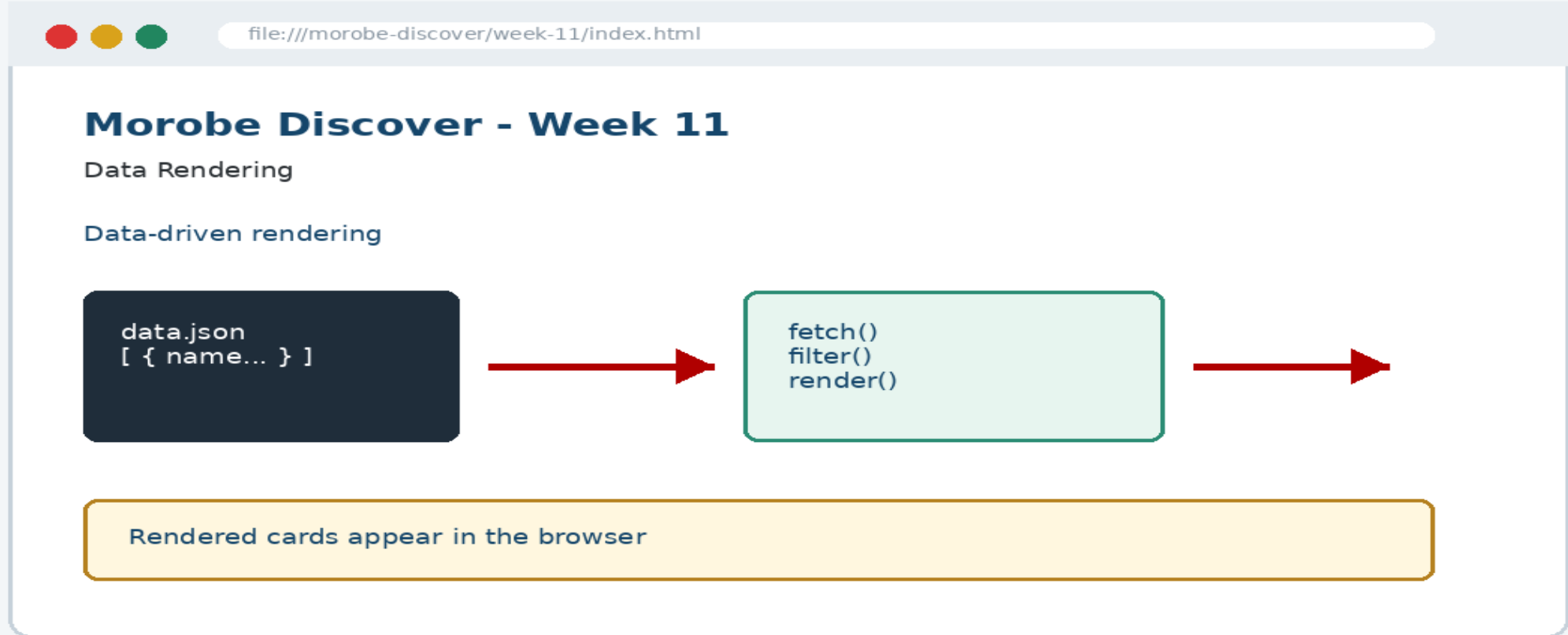
Concept 2

Template strings can create consistent repeated HTML

Concept 3

Empty and error states are part of the interface

Project anatomy



Key code pattern A

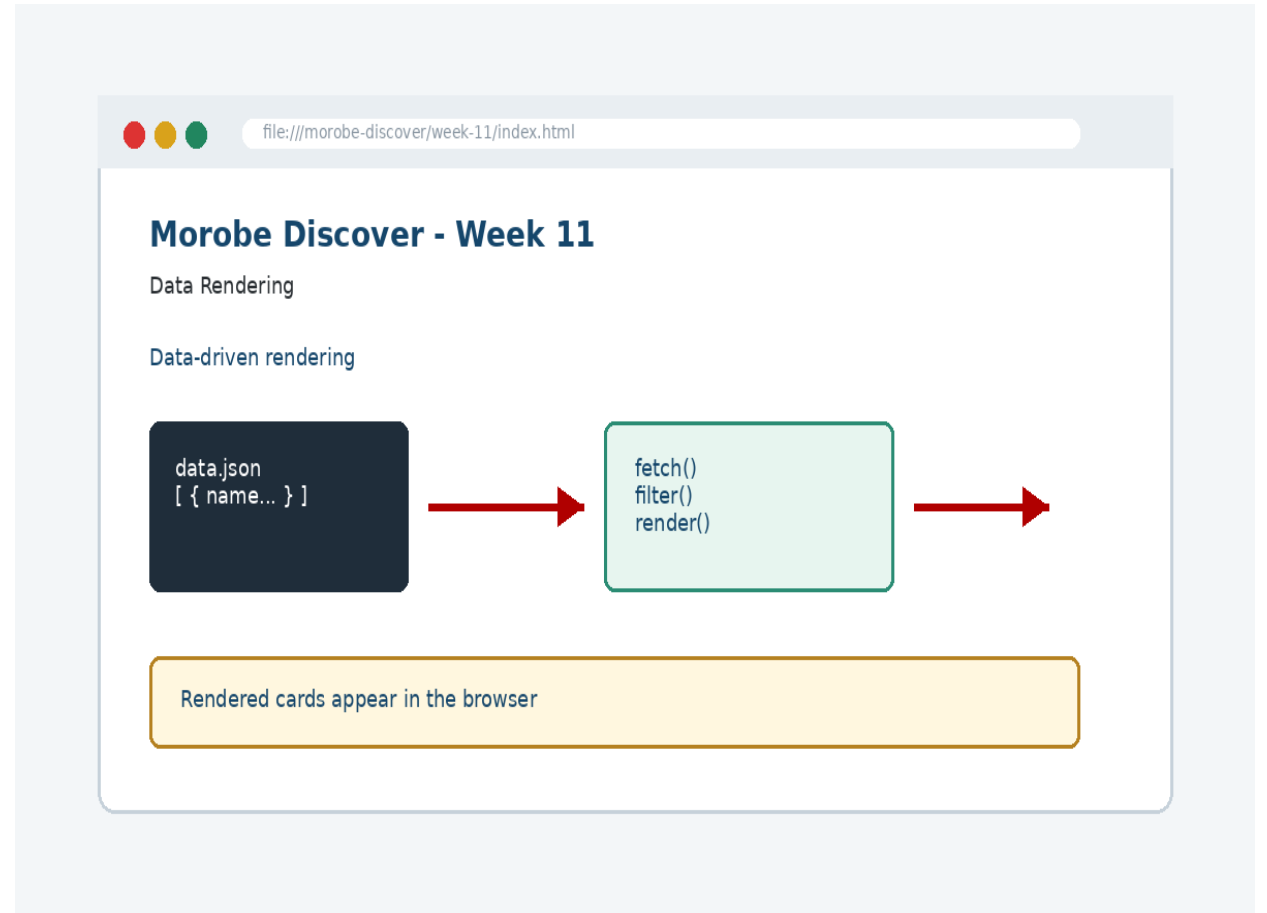
```
const matches = destinations.filter(item =>  
  item.name.toLowerCase().includes(query)  
);
```

Explain it

Which file contains this code?
Which line creates the visible change?
What should appear in the browser?

Code -> visual result

```
const matches = destinations.filter(item  
=>  
item.name.toLowerCase().includes(query)  
);
```



What changed?

Project file changed for Data Rendering

The browser output is now closer to the required weekly evidence

The change supports a user task or quality requirement

The change can be tested and committed

Key code pattern B

```
results.innerHTML = matches.length  
  ? matches.map(renderCard).join("")  
  : "<p>No destinations found.</p>";
```

Why this pattern is needed

The second pattern usually applies the same idea to a different component, state, layout or data flow.

Apply it to Morobe Discover

Review the data shape
Write renderCard() for one item
Render all matching items
Add filter controls and empty states



Compare the approaches

Weak approach

Memorise the definition
Copy code without understanding
No browser evidence
No test note

Professional approach

Explain the concept
Apply the correct code pattern
Inspect the result
Commit and describe the evidence

Common mistake

Mistake

The page appears unchanged, but the student assumes the feature is complete.

Correction

Check the correct file, refresh the browser, inspect DevTools, and verify the expected output.

Why does this fail?

```
// Broken checkpoint  
// File changed, but the browser result does not  
change.  
// Possible causes:  
// - wrong file  
// - file not linked  
// - syntax error  
// - browser not refreshed
```

Class question

Which cause would you check first, and why?

Fix the problem

- Confirm the browser is opening the correct file
- Check links to CSS/JS/data files
- Inspect the console for errors
- Use a small working example
- Test again before committing

Class activity

Activity - 7 minutes

Trace one destination object from JSON file to rendered card.

Work in pairs

Point to the project evidence

Prepare one answer to share

Implementation steps 1-3

Review the data shape

Write renderCard() for one item

Render all matching items

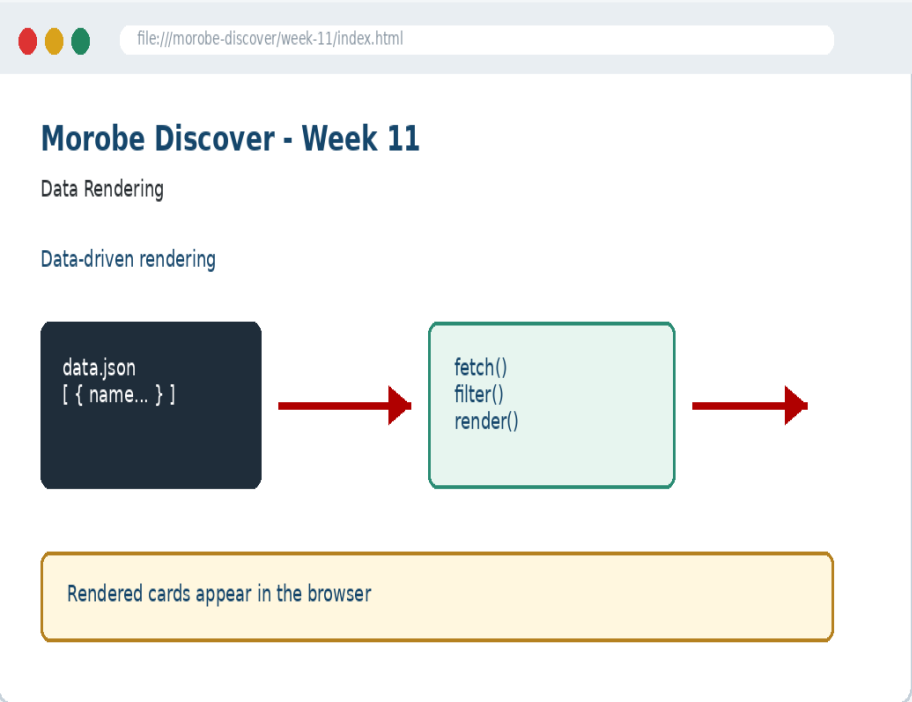
Implementation steps 4-5

Add filter controls and empty states
Connect favourites to rendered cards

LIVE DEMO

Demo checklist

1. Open the current project
2. Locate the affected file
3. Add or modify the key code
4. Refresh the browser
5. Inspect the result
6. Explain the change



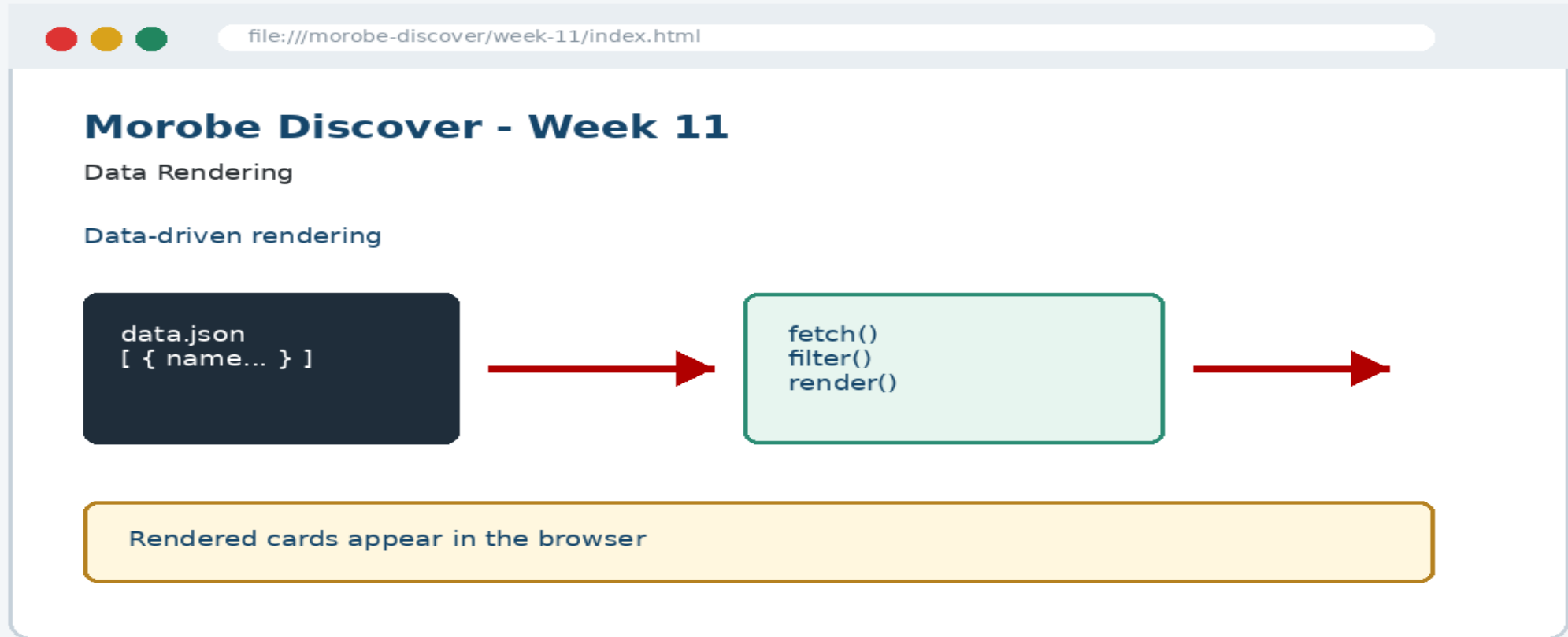
Predict the output

```
results.innerHTML = matches.length  
  ? matches.map(renderCard).join("")  
  : "<p>No destinations found.</p>";
```

Prediction prompt

What should change on screen when this code is applied correctly?

Browser result



Testing the feature

Search for a known result

Search for no result

Disable/rename the JSON file

Confirm the UI explains loading and errors

Inspect with browser tools

Inspection path

Right click -> Inspect
Elements panel -> structure/classes
Console -> JavaScript errors
Network/Application -> data/storage when relevant

Evidence to save

Screenshot
Short test note
Changed files
Git commit message

Git evidence

```
git add .  
git commit -m "Week 11 Data Rendering project  
increment"
```

Why it matters

Git history shows progression from Week 01 to final deployment. It is part of the assessment evidence.

Complete source code

Repository:
`https://github.com/YOUR-USERNAME/is229-morobe-discover`

Weekly folder:
`/week-11/`

Complete project:
`/complete-project/`

Reminder

Do not paste a whole file into a slide. Use the repository for complete code.

Mini challenge

Challenge

Modify one part of the Week 11 implementation and explain what changed in the browser.

State the file edited

State the line/section changed

Show the before and after result

What counts as evidence?

Technical evidence

Changed code files
Browser screenshot
Test note
Git commit

Understanding evidence

Code explanation
Why this method fits
Connection to previous week
Connection to next week

Project lab handoff

Review the data shape

Write renderCard() for one item

Render all matching items

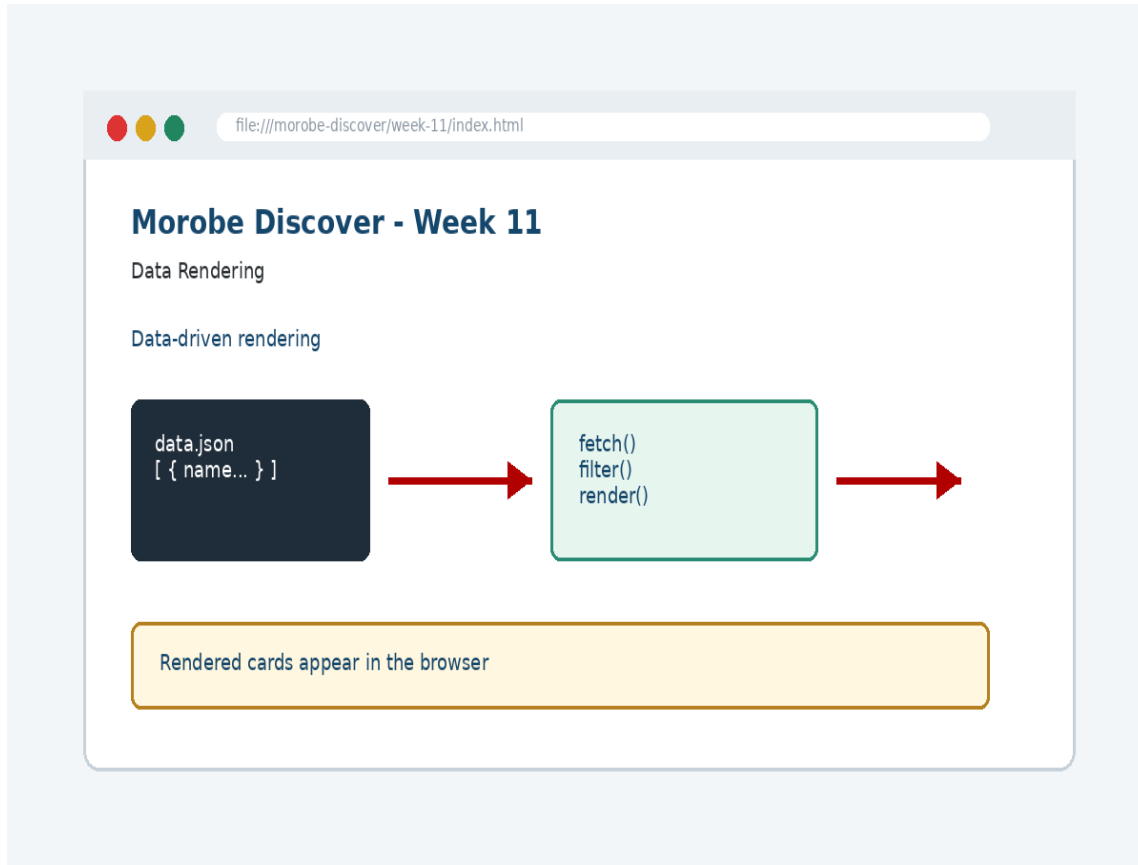
Add filter controls and empty states

Connect favourites to rendered cards

Definition of done

Week 11 feature is implemented
Project still preserves earlier week features
Browser result matches the expected output
Test note and screenshot are saved
Git commit message is meaningful

What we built today



The site renders data-driven content more reliably and connects filters to the user interface.

Evidence: JSON content, fetch requests and filters

Technology: JSON / Fetch API

Lecture summary

Data rendering is a pipeline: fetch, validate, transform, render and handle failure.

Arrays can be filtered before rendering

Template strings can create consistent repeated HTML

Testing and Git are part of development, not extras

Prepare for next class

Finish this week's lab before the next class
Read the next topic in advance
Bring the current project ready to extend
Be ready to explain one code decision

Exit question

How did Week 11 improve Morobe Discover, and what will the next week build on?